

SID Ripper Pro

By Milasoft

April 19, 2026

Introduction

SID Ripper Pro consists of two separate programs. It's designed to assist the user in finding music embedded within Commodore 64 software. To my knowledge, this is the most advanced music ripper available for the Commodore 64.

Requirements

Because the cartridge image relies on the FC3 bank-switching logic, you must use a modern flash solution that correctly emulates Type 3 CRTs (like the Kung Fu Flash, EasyFlash 3, or Ultimate 1541-II/II+). Older generic EPROM boards without the latch logic will freeze on boot.

The Commodore 64 CRT version

The C64 cartridge version is designed to be in memory all of the time. It's best used when you don't have a means to reset your C64 (ie. you don't have a reset cartridge or don't want to short the user port pins) or when you're using an emulator. It has the advantage to always be present but not active until you require it.

Due to limitations, only memory between \$0800 to \$C000 is scanned. Cartridge RAM (\$8000-\$9FFF) is occupied by the cartridge. Use this program for ripping music out of intros, demos, etc.

The CRT has only been tested on Vice and CCS64, not actual hardware.

The Commodore 64 PRG version

The C64 program version is designed to reside in memory exactly when you load it. As this obviously overwrites existing RAM, its primary purpose is to find music under the ROM (\$A000-\$FFFF) which the cartridge version would have otherwise masked. Naturally you'll have to perform a **soft-reset** on the game to get to the point you could perform a disk load of this file. A soft-reset retains the memory when the machine reboots.

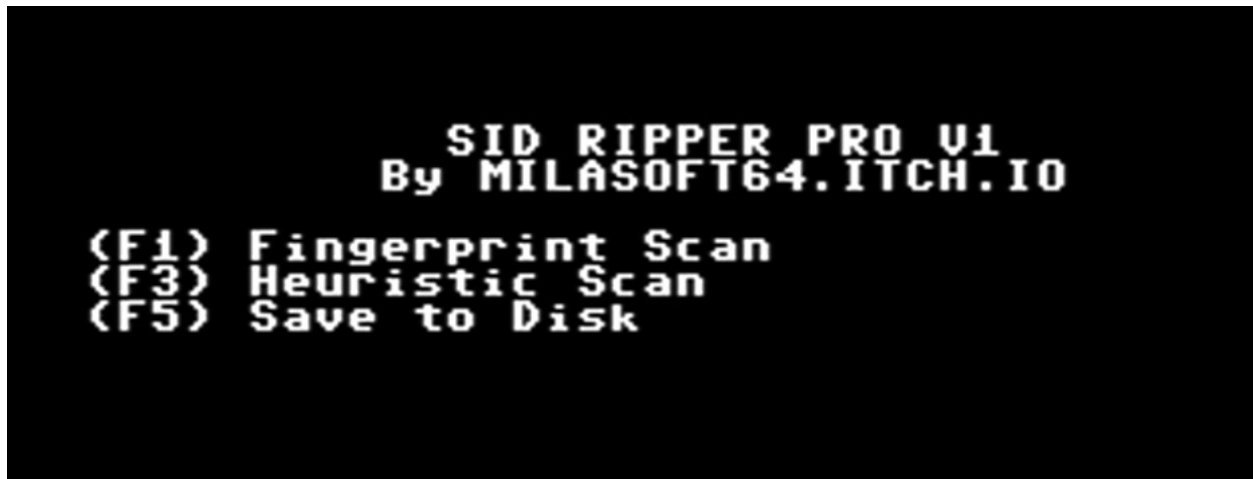
In other words, the CRT version will work for music in the areas between \$0400 to \$7FFF while the PRG ripper will work for areas between \$5000 to \$FFFF. See note below.

Author's Note:

I wanted to create just the cartridge version that would scan all of the memory including under ROM, but the banking process continued to cause crashes. In order to read \$8000 through to \$FFFF it was necessary to use RAM routines when the cart had to be turned off. After a week of non-stop crashing, I abandoned the single-cartridge idea. It just didn't work out.

Quick Start Guide for Cartridge

- 1) Attach cartridge image.
- 2) Load program with the desired SID music.
- 3) Press Freeze button.



Quick Start Guide for PRG

- 1) Load program with the desired SID music.
 - 2) Soft-reset the machine (ALT-F9 in Vice, ALT-R in CCS) or reset switch on real hardware.
 - 3) Load SID Ripper Pro.
-

Fingerprint Scan (F1)

The fingerprint scan is the primary and most reliable method. It works by scanning RAM for known "anchor" instructions — specific SID register writes (STA \$D418, STA \$D404, etc.) that appear in almost every SID player. When it finds one, it cross-references it against an internal table of known SID player fingerprints built from music players. If the anchor matches a known fingerprint, it walks forward to find the JSR that calls the music player and extracts the PLAY and INIT addresses.

A **green** result means the match was fully confirmed — the anchor was found, the fingerprint matched, and the JSR target was verified in RAM.

A **red** result means the anchor was found and the best candidate was identified, but it couldn't be fully verified against the fingerprint table. It may still work, but treat it with less confidence.

Once a SID is found, you may (S)ave memory to disk or (C)ontinue scanning further memory.

Heuristic Scan (F3)

The heuristic scan is the fallback method for games the fingerprint table doesn't know about. Instead of looking for known patterns, it traces the game's actual IRQ chain — following the \$0314/\$0315 and \$FFFE/\$FFFF vectors through every raster interrupt handler the game installed, enqueueing every subroutine each handler calls. It then scores each candidate by counting how many unique SID registers it writes to, directly or indirectly. The highest-scoring candidate that was called from the IRQ chain wins.

Because it works from first principles rather than a lookup table, it can find music in games the fingerprint scanner misses — but it can also be fooled by game logic that happens to touch SID registers for non-music reasons (sound effects, volume fades, etc.). Always listen-check the result.

The heuristic result is shown in **cyan** to distinguish it from fingerprint results.

Save to Disk (F5)

Opens the hex viewer pre-loaded with the found PLAY address. Use **S** to set the start address, **E** to set the end address, then **F7** to save. The file is saved with the name `xxxx-yyyy` where `xxxx` is the start address and `yyyy` is the end address.

Example: 0800-3000.prg

```
SID RIPPER PRO - MATCH FOUND
PLAY:$1000      INIT:$1000
```

```
-----
START:$1000     END:$1100
-----
```

```
$1000:  a9 50 85 5e a9 04 85 5f  P . . .
$1008:  a9 2d a0 27 91 5e 88 10  - . . .
$1010:  fb a9 06 a0 27 99 50 d8  . . . P .
$1018:  88 10 fa a9 a0 85 5e a9  . . . .
$1020:  04 85 5f a9 2d a0 27 91  . . . .
$1028:  5e 88 10 fb a9 06 a0 27  . . . .
$1030:  99 a0 d8 88 10 fa a9 70  . . . .
$1038:  85 5e a9 07 85 5f a9 2b  . . . .
$1040:  a2 14 20 3b 13 a9 98 85  E . . .
$1048:  5e a9 07 85 5f a9 47 a2  . . . .
VIEW:$1000
```

```
[F1/F3]=PAGE  [L]=START  [G]=GOTO
[S]=SET START  [E]=SET END
[YY]=SAVE
```

The process is not as complex as it may appear. Use F1/F3 to page through memory to get a visual idea of where the music will end (as you know where it begins). S and E keys define the start and end points for the disk save. 'G' will go to an exact address. As the CRT version does not save under ROM, the RAM areas of \$A000-\$BFFF and \$E000-\$FFFF are not visible.

The EXE version

This is the most powerful of the tools. The Windows x64 executable file is designed to perform a variety of functions:

```
+-----+
| C64 SID Ripper Toolkit |
| DB: sid_fingerprints.pkl |
+-----+
| 1. Scan a VSF snapshot |
| 2. Build fingerprint database |
| 3. Validate scanner vs SID corpus |
| 4. Show corpus offset stats |
| 5. Quit |
+-----+
Choice:
```

1) Scan a VSF snapshot

This is the primary function, which will parse a Windows VICE Emulator VSF freeze file. It will follow the chain of interrupts (IRQ and NMI) to deep-seek calls in search of the music routine.

This is more extensive than using a machine language monitor to search for music.

```

1. 1581.vsf
2. cobra6.vsf
3. lightforce.vsf
4. outrun.vsf
5. rambo15.vsf
6. vice-snapshot-20260419174747.vsf
0. Enter a path manually
Pick number (0 to type path): 3

Loading lightforce.vsf ...
VSF: offset 183, preamble=4b  IRQ=$EA31
VSF: IRQ=$EA31  NMI=$FE47  HW_IRQ=$C7F1
Checking fingerprint database...

Fingerprint matched 'A True Story' but validation failed:
  VSF PLAY=$5F19  SID score=0/0 (need >=4/2)
  VSF INIT=$5EF9  first byte=$00  init_ok=False
Treating as false positive -- falling through to scanner.

No fingerprint match - running scanner...

Building seeds...
$EA31  IRQ vector ($0314)
$FE47  NMI vector ($0318)
$C7F1  HW IRQ ($FFFE)
$C6E0  HW NMI ($FFFA)
Scanning for IRQ install patterns ($0314/$0315, all register variants)...
Scanning for CIA timer IRQ setup ($DC0D)...
(no CIA timer IRQ patterns found)

[Phase 1] Tracing...
111 traced, 13 to score

[Phase 2] Scoring for SID writes...
$EA31  total=1  direct=0  depth=0
$C7F1  total=1  direct=1  depth=2
$E168  total=1  direct=0  depth=4
$E0E6  total=1  direct=1  depth=5
$E208  total=1  direct=0  depth=6
$F0BF  total=3  direct=3  depth=10

[Result] Top candidates:
$F0BF  total=3  direct=3  [PLAY]
$C7F1  total=1  direct=1  [PLAY]
$E0E6  total=1  direct=1  [PLAY]
$EA31  total=1  direct=0  [wrapper/handler]

[INIT] not found - single tune or init unknown
To select: JSR $F0BF (no tune parameter needed)

[Enter to continue]

```

There is no guarantee the results will be correct and you may find yourself without the Init address and only a Play address.

2) Build fingerprint database

When combined with the latest copy of the HVSC SID database, this option creates a corpus (a database collection) of the offsets required. In plain English, it examines unique signatures found inside the player routines and measures the distance between them and the Play and Init routines. The C64 version works the same way.

3) Validate scanner vs SID corpus

This will scan your HVSC folder(s) and compute a percentage of how many fingerprint matches it can recognize.

```
--- Validate Scanner Against SID Corpus ---
Path to SID collection: c:\c64music
Max files (Enter = all):
Show per-file results? (y/N):
Testing 121146 files...

Loading fingerprint database...
DB has 46777 fingerprints
500/121146 accuracy: 99.0% fp_matches: 498
1000/121146 accuracy: 98.4% fp_matches: 998
1500/121146 accuracy: 98.8% fp_matches: 1497
2000/121146 accuracy: 98.6% fp_matches: 1997
2500/121146 accuracy: 98.9% fp_matches: 2497
3000/121146 accuracy: 98.9% fp_matches: 2997
3500/121146 accuracy: 98.9% fp_matches: 3496
4000/121146 accuracy: 98.9% fp_matches: 3996
4500/121146 accuracy: 98.9% fp_matches: 4495
5000/121146 accuracy: 98.9% fp_matches: 4994
5500/121146 accuracy: 98.9% fp_matches: 5494
6000/121146 accuracy: 98.9% fp_matches: 5994
6500/121146 accuracy: 98.8% fp_matches: 6494
```

Terms:

Init address – the address that you call to reset the music to the beginning. Often performed by setting A to a value of 0. Multiple-tune music would use values 0,1,2.. etc.

Play address – the address that you would call to play the music.

Questions

Q: Will it work with every program?

A: No. <- Read that again. No, it won't. Ripping music is not a novice task.

Q: What's the success rate?

A: That depends on the program. Success will be greater for 'scene' productions over commercial games. The base signatures target approximately 40,000 SID files.

Q: How do I know where a SID file ends?

A: You won't know, however a hex viewer is provided to assist you with a visual display. Also note that if music begins at an even bank such as \$1000, it most likely ends by \$1fff (but not exclusively).

Q: SRP found the play address but not the initialization address.

A: Success in finding the Init address routine varies. It could be requiring a value of #\$01 into a location. It could be a JSR just above the play routine. It could be found later in the player code. There's no "one way" to find it other than to trace through the code-flow of the game/intro/demo. It will normally occur BEFORE the IRQ is set up.

Q: Is there a built-in music player?

A: Not presently, including one would have been a sure way to crash false positives 😊

Q: Will there be updates?

A: If the demand is there, I may release a 100% CRT version with visualizer and PSID creator or you may never see another version.

SID Ripper Pro is provided for personal, educational, and archival use only.

Music ripped using this tool may be subject to copyright. You are responsible for ensuring your use complies with applicable laws in your jurisdiction. The author makes no claim of ownership over any music extracted using this software, and accepts no liability for how extracted content is used or distributed.

This tool is intended for preserving music from software you own, studying SID player techniques, or contributing to archival projects such as the High Voltage SID Collection. It is not intended to facilitate copyright infringement.

The software is provided as-is, without warranty of any kind. The author accepts no responsibility for data loss, hardware damage, or any other consequence arising from its use.

Milasoftware

Milasoftware64.Itch.io

High Voltage SID Collection

<https://www.hvsc.c64.org/>